

Full length article

# Semantic web-based automated compliance checking with integration of Finite Element analysis

Panagiotis Patlakas<sup>a,\*</sup>, Ioannis Christovasilis<sup>b</sup>, Lorenzo Riparbelli<sup>b</sup>, Franco KT Cheung<sup>a</sup>, Edlira Vakaj<sup>a</sup>

<sup>a</sup> Faculty of Computing, Engineering, and the Built Environment, Birmingham City University, Birmingham, UK

<sup>b</sup> Aether Engineering, Florence, Italy



## ARTICLE INFO

### Keywords:

Automatic Compliance Checking  
Web Ontology Language (OWL)  
Industry Foundation Classes (IFC)  
Semantic Web  
Finite Element Analysis (FEA)

## ABSTRACT

Automatic Compliance Checking (ACC) is a promising response to the challenges involved in meeting building and planning regulations, and increasingly utilised by researchers in the context of Building Information Modelling (BIM) and the Industry Foundation Classes (IFC). However, engineers often use computational methods, such as Finite Element Analysis (FEA), to solve challenging engineering problems. Such methods have received little attention in the context of ACC and BIM. The work presented here proposes a holistic approach, integrating FEA in the IFC schema, and utilising a Semantic Web-based system for ACC. A top-level ontology, the Building Compliance Ontology (BCO) is proposed, with the aim of covering all aspects of compliance in the Built Environment. One novelty of BCO is that it revolves around modelling statements in the building regulations, as opposed to the more common approach of modelling building components.

In order to evaluate the effectiveness and applicability of this process, a real-world problem is used: the compliance to the structural engineering design codes of the structural members of a modular steel structure. An open-source solver is utilised for the FEA, and the IFC model is enriched with its results, using an open-source middleware for the Input/Output. BCO is extended to the field of structural engineering, and appropriate ACC classes and SHACL checks are implemented. Compliance is checked automatically by running SHACL scripts in a Semantic Web environment.

## 1. Introduction

Ensuring compliance to the planning and building regulations is an essential part of the building design and construction process as compliance failure can have a substantial impact, from financial (e.g., resulting in delays or need for amendments), to catastrophic, (e.g., resulting in building collapse and serious accidents involving loss of life). Checking compliance with the regulations is a complex and time-consuming process, and therefore automating this process has been a long-standing ambition in the Architecture, Engineering, and Construction (AEC) sector.

Automatic Compliance Checking (ACC), sometimes referred to also as Automatic Code Compliance, has an extensive history going back over 50 years [5]. While the definition has evolved over the years, the fundamental concept is the automatic verification of building design compliance with relevant codes, regulations, and standards. In some cases, this definition is expanded to cover contractual aspects, or more

generally, compliance with client stipulations and extended to the whole lifecycle [28]. The common expectation is that this automation will reduce human errors [11] increase efficiency, lower costs, and improve construction project quality, ultimately contributing to safer, more sustainable built environments [37]. A comprehensive investigation of the state-of-the-art in ACC by Amor and Dimyadi [1] suggests there is considerable potential, but also identified substantial challenges in quality assurance, databases for the validation of ACC processes, and inherent challenges in translating regulations consistently. It also identified particular challenges in two areas: the use of performance-based codes, which would typically require the use of advanced simulation tools, and the quality and information of the building models utilised. The underlying assumption is that the standard building model used will rely on Building Information Modelling (BIM).

This assumption is understandable, as in the past fifteen years BIM has emerged as the de facto approach for research into the digital aspects of AEC. While competing definitions of BIM have appeared [17], a

\* Corresponding author.

E-mail address: [panagiotis.patlakas@bcu.ac.uk](mailto:panagiotis.patlakas@bcu.ac.uk) (P. Patlakas).

<https://doi.org/10.1016/j.aei.2024.102448>

Received 16 June 2023; Received in revised form 24 December 2023; Accepted 26 February 2024

Available online 14 March 2024

1474-0346/© 2024 The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

popular one suggests that BIM is a process for creating and managing information about a built asset utilising digital technologies[14]. Typical assumptions of BIM are that these digital technologies will be multi-disciplinary, shared, structured, information-rich, and operating in 3D [2,6,15]<sup>1</sup>. In the AEC industry, this can often be interpreted as the use of specific BIM software. However, in AEC research, this usually implies the use of the Industry Foundation Classes (IFC) data schema.

Researchers have used BIM as the system for ACC for anything from planning[22], to structural engineering[23], to fire safety[18], to environmental pollutants[39]. Some key challenges in these processes involve the complexity of the regulations, as well as the successful interpretation of those in a format computers can interpret [38]. As a result, more recently Semantic Web approaches, including development of domain-specific ontologies, and Natural Language Processing (NLP) technologies, for automating the development of digital models of the Building Regulations texts, are becoming increasingly popular [16,20,40]. Peng and Liu [25] combined ifcOWL, the Web Ontology Language (OWL) representation of the IFC, with knowledge graphs, in order to create a human-machine-readable pattern that can lead to automatic compliance checking of simple problems.

The inherent challenge of combining such complex and diverse technologies means that relatively little attention has been paid to solving more computationally-intensive problems. Researchers often look to automate the checking of relatively straightforward provisions, where the data that is checked against in the regulations is provided in the model, and is typically a geometric property, such as the height of a wall, or a Boolean condition, such as the provision (or not) of a smoke alarm. A subset of Building Regulations fulfils these conditions, especially when specialist engineering input is not expected. Even then, however, this is only a small percentage. For example, looking at Part B1 of the England and Wales Fire Safety Regulation, Malsane et al [18] identified only 27 declarative clauses out of 137 clauses in total, where declarative is defined as directly computer-interpretable.

More complex regulations, such as structural design codes, have few clauses that fulfil such conditions and rely on a high degree of expert interpretation. More importantly, the properties that are meant to be checked against are not properties directly derived from a model, but the product of complex calculations and simulations themselves. Automatic Compliance Checking in such cases involves not only checking the submitted drawings or models, but the intermediate calculations performed during the design process and the scientific assumptions made. The past three decades, these intermediate calculations are often the output of computationally-intensive simulation software, based on techniques such as Finite Element Modelling (FEM) and Computational Fluid Dynamics (CFD), which use different data structures and modelling assumptions, outside the IFC realm.

The work presented here aims to propose and evaluate a novel, comprehensive process for the ACC of structural engineering regulations, that combines FEM, for the simulation of structural engineering performance, IFC, as a common data exchange format, and semantic web technologies for ACC. It introduces a proprietary tool for the interaction between an open-source FEM solver, Code Aster, and the IFC file format; it presents a middleware Python-based tool for the interaction between the IFC file and the semantic web-based compliance environment; finally, it introduces a statement-based ontology for the verification of building regulations compliance. This approach has the potential to significantly streamline and automate the compliance checking process in the field of structural engineering at first instance, and is generalisable to all aspects of the Built Environment.

The proposed process is applied in a case study involving modular construction in the UK. This type of construction is generally regarded as having a range of benefits compared to traditional on-site construction, including improving quality and efficiency, and reducing construction

time and waste [4,36]. Modular construction lends itself to ACC particularly well, as the standardised modules it depends on reduce the complexity and “one-off” design cases often found in traditional construction. Moreover, the offsite manufacturing process already involves a digital element, which can be integrated with Semantic Web approaches [31]. At the same time, it is a sector whose considerable potential has largely not been realised, in part due to incompatible digital workflows [34].

While the work presented here focuses on structural engineering, the workflow and, particularly, the ontology-based approach proposed, are potentially generalisable to all aspects of Automatic Compliance Checking. FEM, together with CFD, are arguably the most computationally-intensive simulation approaches employed in all construction and, therefore, extension of this approach to other areas of regulatory compliance should come with lower or equal complexity. Moreover, the high-level ontology presented has been developed with the intention of accommodating *any* type of building regulation, while the structural engineering-specific classes that verify compliance are just one possible area of application.

Section 2 provides the compliance problem description in Structural Engineering, and suggests a schematic, generic solution, based on a multi-platform workflow. Section 3 describes a top-level Building Compliance Ontology (BCO), whose aim is to accommodate implementation of any type of Building Compliance regulation. Section 4 provides an implementation of the workflow in the Structural Engineering domain. This includes identifying specific FEM-based technologies and the Input/Output (I/O) issues between them, implementation of BCO for Structural Engineering with application-level implementation of some Eurocode 3 provisions for member checks. Section 5 presents the application of this workflow to a specific case study. Finally, Section 6 concludes the work, providing a brief discussion of the potential of this approach, the limitations of its current application, and the challenges that need to be overcome for its generalisation.

## 2. Problem Description and solution concept

### 2.1. Structural engineering and the compliance process

Structural Engineering is one of the most vital, and scientifically most complex, aspects of building design. Building Regulations worldwide emphasise the importance of structural integrity, and the submission of detailed reports, completed by appropriately qualified and experienced structural engineers is typically expected. While simplified calculation processes are at times given for simple buildings, such as the UK's Approved Document A, the standard expectation is to demonstrate compliance to advanced structural design codes. In most European countries, the de facto structural design standard is the Eurocode design suite, with de jure recognition in some jurisdictions. In line with other contemporary design codes worldwide, the Eurocodes combine a wide range of provisions, from providing specific numerical checks that need to be satisfied (e.g. that a certain force on a structural member is lower than the member's resistance), to specification of how some real-world aspects should be modelled (e.g. the loading values that should be taken for residential floors), to guidance on general modelling principles of physical phenomena (e.g. the type of structural analysis that should be undertaken for a structure). These design codes assume an expert readership, intentionally allow substantial room for interpretation, and are not self-contained, i.e. they assume complex relationships with other regulation documents, such as European Norms, European Technical Assessments, and manufacturers' literature (e.g. for establishing basic values).

The structural design process in practice is similarly complicated. The structural analysis and design process affects, and is affected by, not only the architectural design, but the rest of the design process. Fig. 1 provides a typical workflow.

The interactivity, and non-linearity, of this process should not be

<sup>1</sup> IFC2CA Git repository: <https://github.com/Jesusbill/ifc2ca>.

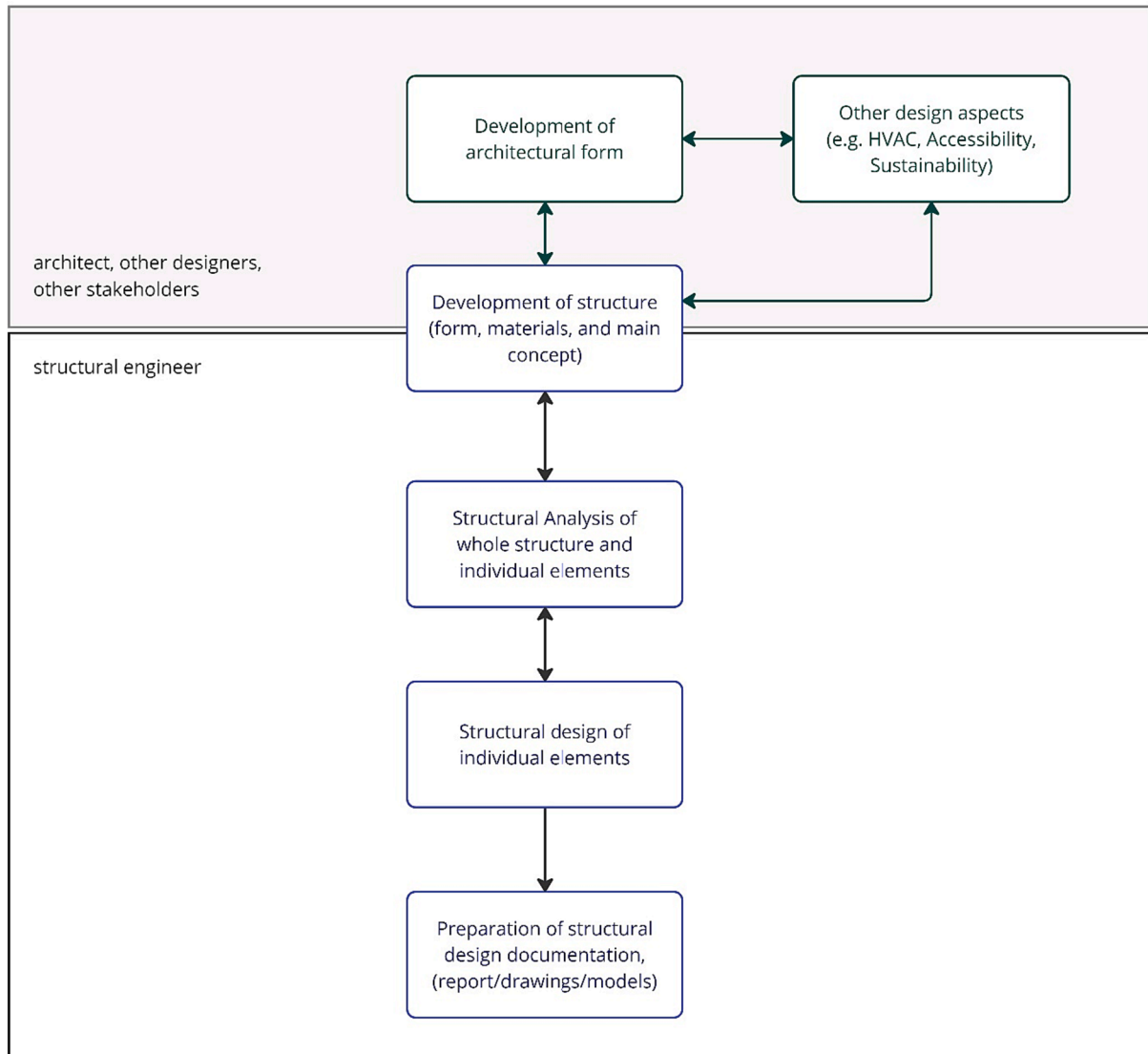


Fig. 1. Sample structural design process and relationship with the overall design.

underestimated: the double arrows in Fig. 1 signify that design in each step can trigger a reconsideration of the steps before it. This is particularly important in interactions between the structural engineer and the rest of the designers. Changes in one part of the process (e.g. Building Services) can impact both the structural design and the overall process. Lack of communication, particularly in big projects with multiple stakeholders, can have significant impact.

BIM is meant to address this, by providing a common model format, such as the IFC, which, theoretically, enables all designers to be working from the same model. In practice, this is not so straightforward. Most designers employ specialist software, whose integration of BIM is rarely seamless. This is particularly relevant when computationally intensive software is utilised, such as FEM software for structural analysis. It is not unusual for each node of the diagram in Fig. 1 to require its own software, e.g. one package for the structural analysis, at least one for the structural design (as specific elements, such as complex connections, might require different software to the design of members), and a different package for the preparation of structural documentation. Compliance with the Building Regulations is relevant to all three aspects. However, most software packages will focus on demonstrating compliance in the structural design stage. Expert knowledge is typically assumed for following the right structural analysis process and adhering to construction requirements, such as detailing aspects.

Two main challenges derive from this process. Firstly, there is considerable Input/Output (I/O) required between the different nodes of the process. This can often include manual development of different models, especially if complex simulation engines such as FEM are employed. A second challenge comes from the type of Building Regulations to be checked: these are often siloed in different software packages involved in each step of the process, without central organisation, and limited automation. It is generally expected that a high degree of expertise is involved in the process, with the overall responsibility resting with the designer. From a regulator's perspective, meanwhile, the responsibility is meant to rest primarily with the designer, with limited verification of the design taking place. The accompanying documentation is not intended so much as a document that will be utilised to verify compliance (except in limited cases, in some jurisdictions), but primarily as a record that can be reviewed if structural failures are identified in the future.

## 2.2. A holistic automated building compliance checking process

The challenges described in the previous sub-section are not particular to Structural Engineering. If viewed in abstraction, they can be generalised in many aspects of building design. These key challenges are summarised in Table 1 below.

**Table 1**

Challenges in Structural Engineering and other AEC domains, and their generalisation.

| Structural Engineering Challenge                                                                                                                                                                                                                                                         | Similar Challenges in other AEC domains                                                                                                                                                                                                                                                                                                                                                                                                                 | Generic Challenge                                                                                                                                              |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FEM-based analysis is often required for complex structural analysis, requiring computationally-heavy software.                                                                                                                                                                          | CFD-based analysis can be required for environmental analysis, such as ventilation, thermal comfort, or acoustics requiring computationally heavy software [21]. CFD-based analysis can similarly be used for simulations of fire behaviour, as employed by fire engineers [8]. Other complex mathematical models are utilised by building designers for specialist cases, e.g. evacuation modelling, again relying on specialist software [32].        | Complex analysis, of a computationally-heavy nature, has to be performed on separate software, outside a BIM authoring tool.                                   |
| Specialist elements of structural design, such as proprietary structural systems or design in uncommon materials (e. g., outside steel and concrete), often requires specialist software.                                                                                                | Almost all AEC disciplines are called to utilise specialist software, or software linked to a proprietary technology, at certain instances, with limited or no BIM support. Examples include: Architects using generative design tools. Any type of engineer working with proprietary systems, for example in Building Services. Designers working with specialist presentation software. Construction managers working with project planning software. | Specialist software, or software linked to proprietary technology, needs to be utilised, with limited or no BIM support.                                       |
| Maintaining a shared BIM file for the entire AEC chain results in substantial I/O challenges, between the specialist structural analysis and design packages and the common BIM authoring tools, requiring human intervention and increasing the possibility of errors and clashes [29]. | The same challenge applies for any AEC professional working with any type of specialist package without native support for a popular BIM authoring tool.                                                                                                                                                                                                                                                                                                | Maintaining a shared BIM file for the entire chain, in a common BIM authoring tool, relying on I/O between different software packages and the main tool [30]. |
| In order to minimise the I/O described above, structural engineers are incentivised to work with commercial software orbiting around the most successful commercial BIM authoring tools, effectively being tied to commercial data schemas.                                              | The same incentives apply for any AEC professional working with any type of specialist package without native support for a popular BIM authoring tool.                                                                                                                                                                                                                                                                                                 | Reliance on commercial data schemas and proprietary file formats.                                                                                              |
| The regulator, the client, and other stakeholders have no straightforward                                                                                                                                                                                                                | The same applies for most other types of engineering design that                                                                                                                                                                                                                                                                                                                                                                                        | Regulators, clients, and other stakeholders (including other                                                                                                   |

**Table 1 (continued)**

| Structural Engineering Challenge                                                                                                                                                                                                                                                                    | Similar Challenges in other AEC domains                                                                                                                                                             | Generic Challenge                                                                                                                                                                                                                                                                              |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| way of verifying the results of the structural analysis and design. There is an assumption of the building being compliant, based on the structural engineer having signed it off. The accompanying structural documentation is intended as a record in case of a problem identified in the future. | needs to be compliant to some regulation or standard. Exceptions to that are often regulations relating to architectural form (e. g., coverage of a plot of land), which tend to be easy to verify. | designers) have no direct way to validate and verify the results of the analysis of other professionals. The working assumption is that the design is sound and compliant, and the accompanying documentation serves as a record in case of future failures.                                   |
| Structural Engineering is moving towards real-time structural health monitoring technologies [13].                                                                                                                                                                                                  | Most aspects of AEC are increasingly taking into account aspects of operation, facilities management, and building performance, via real-time monitoring and collection of data [19].               | Real-time monitoring of built assets, and assessment of their performance, means that compliance should increasingly be viewed as a challenge not only on the design and construction stage, which finishes on delivery, but as something that applies throughout the built asset's lifecycle. |

The above can be summarised in the following points:

1. While different AEC disciplines might be utilising different technologies and different design, analytical, and computational methods, there are overarching challenges that, in abstraction, apply to most, if not all, disciplines.
2. Data models, solutions, and processes should be decentralized and utilising different tools and technologies where possible. Reliance on a single BIM authoring tool as a principal technology can lead to data silos.
3. Building Compliance should be more open, allowing regulators, clients, and other stakeholders to receive third-party verification, automated where possible.
4. Building Compliance should be viewed as a whole life-cycle process.

The work presented here suggests the following responses to these issues.

Firstly, the conceptualisation of the common digital repository not simply as a BIM model, but as a Digital Twin. While Digital Twins, like BIM, have various interpretation, a common one posits that a Digital Twin is a digital representation of “real-world” entities and processes, which is powered by data, both historical and real-time, and focuses on outcomes, such as the desired performance of a system [10]. In the current state of AEC development, the distinction between BIM and Digital Twin can appear primarily conceptual. Indeed, it has been suggested that Digital Twins will be integrated in the BIM process [3], and existing data schemas such as the IFC can be used. However, even at a conceptual level, the distinction can be important, partly because it suggests whole-life cycle compliance and a reliance on real-time data, and because it can signal a move towards more open-ended processes than BIM has come to signify in industry practice.

Secondly, compliance is decoupled from the data model. A Semantic Web, ontology-based approach for the compliance checking provides greater opportunities for cross-platform integration and handling heterogeneous data. In the future, it could also allow compliance systems to benefit from emerging developments such as Dynamic Knowledge Graphs [35].

One of the most challenging aspects of implementing those concepts



involve working with complex simulation processes, and this is the focus of the work presented here. The case study uses structural engineering for offsite as its specialist domain of application, but the core workflow should, in principle, be applicable to any type of complex numerical simulation, such as other areas of FEM or CFD.

The principal workflow has been conceived in abstract terms, with the ambition to accommodate any type of complex simulation. Specifically:

A Digital Twin model, in IFC or similar format, is developed in a Digital Twin authoring tool.

Complex simulation or calculation processes are undertaken using specialist software. Their results are fed back to the Digital Twin model until a working design is completed.

The Digital Twin model is enriched with the information of the simulation/calculation process.

A middleware is utilised to extract information ACC-related information from the Digital Twin model. The middleware converts this information to ontology instances and exports the resulting ontology in OWL format.

The ontology instances are introduced in a semantic web tool, and compliance is checked automatically for each relevant statement of the Building Regulations

The generalised process is shown in Fig. 2.

The semantic web aspect of this process requires appropriate ontologies to be created to allow automated compliance checking. As building regulations are extremely diverse in style, process, and approach, a high-level ontology is required in order to allow a consistent framework to be

followed. Such an ontology is presented in the next Section.

### 3. The building compliance ontology

#### 3.1. Background

While ontology-based approaches are increasingly becoming popular for AEC compliance checking, many challenges remain. A fundamental issue is the complexity of the Industry Foundation Classes (IFC), often considered the de facto standard for the implementation of Building Information Modelling, at least when open-source approaches are required. It has been observed for some time that IFC contrasts with Semantic Web principles [24] and proposals have been made to modularise ifcOWL to address those issues [33]. An alternative approach not dealing with BIM via IFC, but via domain-specific ontologies, following the concept of Linked Building Data (LBD). An example of this is the Building Topology Ontology (BOT), which covers top-level concepts such as Zone, Site, and Building [26].

When it comes to compliance checking, separate challenges arise from the nature of compliance regulations. Such regulations are typically developed by, and for, subject experts and thus possess a range of characteristics such as:

- semantic ambiguity across disciplines (e.g. what counts as a “column” might be different in a structural engineering design code, and a fire safety one)
- semantic ambiguity within the same document (e.g. an element might be referred to as “beam” in one Clause, but “flexural member” in another)

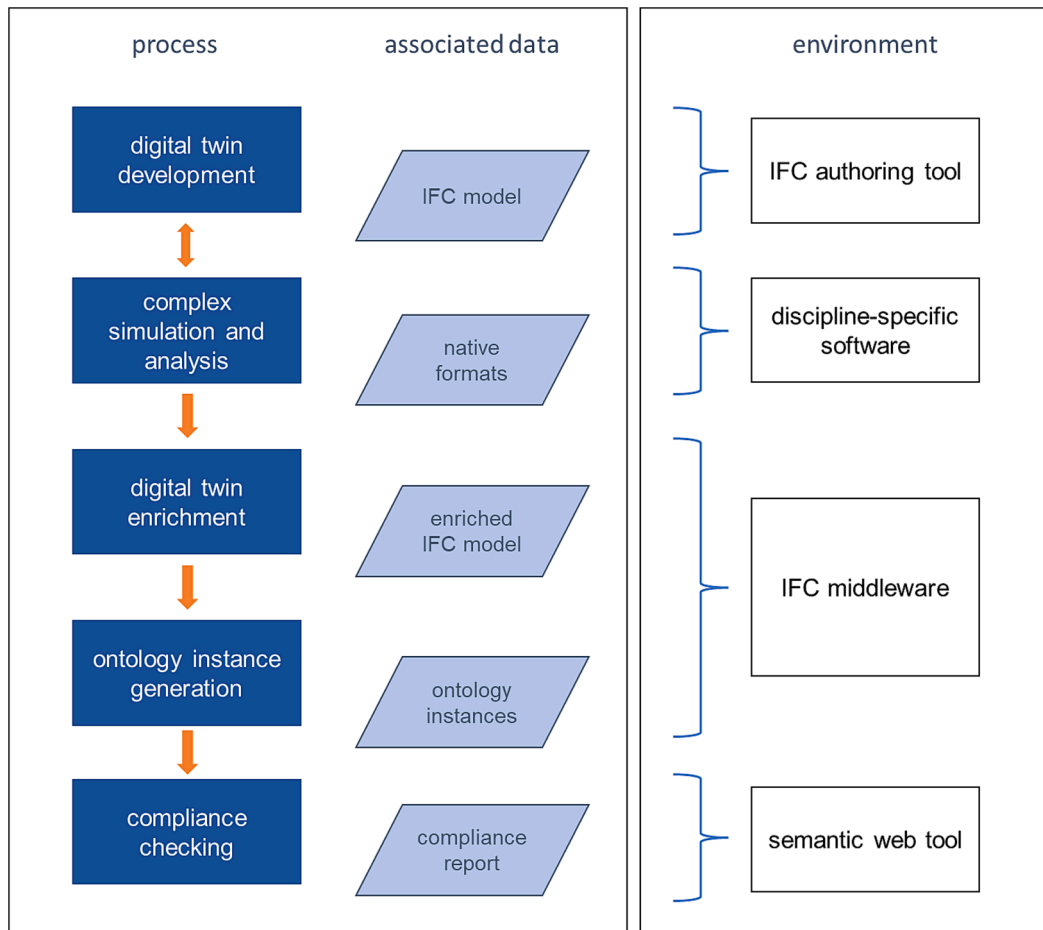


Fig. 2. Generalised process for semantic web-based automatic compliance checking.

- linked or incomplete information. Regulation documents will often contain references to many other documents (e.g. material properties, standardisation documents, etc), or not be self-contained, assuming the reader is a trained professional (e.g. not providing equations for the calculations of concepts taken as given).
- use of concepts that are abstract or open to interpretation (e.g. the same connection might be modelled as rigid in one structural analysis, but semi-rigid in another)

The typical approach for developing AEC ontologies is “bottom up”, i.e. a specific subset of the regulations is identified as a domain, a list of relevant concepts is created, and a specialist ontology is created [31]. Such ontologies are also typically based around building components, i.e. they mimic the analogue method of solving such problems by identifying building components that need to conform to regulations. These components either follow an established BIM ontology, such as ifcOWL, or are developed ad hoc to address a specific problem, such as BOT.

While such approaches are effective when solving specific and small-scale problems within a sub-domain, they can pose issues when a more general framework for ACC is needed, due to the complexities mentioned earlier. The ideal, in line with ontological good practice, is for these specialist ontologies to be interlinked together to form an ecosystem. Yet, linking subdomain-specific ontologies can prove challenging when fundamental concepts (such as a column or beam) do not refer to the same concept across disciplines. Developing a general system capable of accommodating *all* AEC concepts effectively attempts to duplicate the IFC Schema, whose limitations concerning Semantic Web approaches have been described.

### 3.2. Core philosophy

The scope of the proposed ontology, called Building Compliance Ontology (BCO), is substantial. It aims to provide a generic, top-level infrastructure to cover all aspects of Built Environment regulation, across disciplines. Integrated in the framework presented in Section 2, its intended use is to facilitate an Automated Building Compliance Checking process, with clear tracing of the regulations, disciplines, and jurisdictions to which it applies. The intended users, respectively, are AEC professionals interested in compliance, including designers, engineers, permitting officers, and other stakeholders.

BCO has three differences from previous AEC compliance ontologies. Firstly, it is very broad in scope: it does not focus on a single domain, but aspires to provide the high-level capacity to cover *all* types of legislation. Secondly, it is not intended as a stand-alone ontology, but as part of a wider ACC framework. The intention behind this is to utilise a Semantic Web approach where it is optimal, such as the ontological modelling of building compliance regulations, while maintaining the flexibility to integrate complex simulation and computational modelling processes that are better handled with external tools.

The third novelty of BCO is that it is not an ontology based on modelling Building Components, but modelling individual **statements**, primarily compliance checks, on building regulation documents. Instead of attempting to develop generic Digital Twin concepts, it focuses on capturing the information conveyed on the compliance documents. The building information is modelled as *Required Inputs* to the ontology. This approach allows BCO to avoid the challenges stated in Section 3.1, while allowing for much more flexibility in how this data is provided in particular subdomain implementations.

### 3.3. Top-level ontology overview

The core modelling concept of BCO is the **Statement** class, corresponding to *something* described in Building Regulation-related document. The most commonly used **Statement** subclass is **Check**, i.e. a condition that needs to be satisfied to demonstrate compliance, with other core types being **Equation** and **Text**, intended for any other type

of information that does not describe a condition to be satisfied or a mathematical equation. A collection of **Checks** can also be grouped into **CheckLists**.

The core properties of each **Statement** are divided into three categories: properties related to data inputs; the source of the **Statement** to the Building Regulations (i.e. document and specific part of that), AEC-related info (e.g. which discipline and building type this statement refers to, as well as the legal status of this **Statement** in different jurisdictions), and finally information around the permitting process, describing aspects such as what evidence is required to demonstrate compliance, who is responsible for verifying this statement, and to which stage of permitting it pertains.

A graph of the top-level ontology is given in Fig. 3.

Subclasses of **Check** are developed to allow the checking of different types of conditions. **NumericalCheck**, the most commonly employed one, is for verifying conditions around mathematical equations (e.g. is the width of a corridor above a required threshold). **CategoryCheck** allows the validation of checks that involve categories, typically via table look-up functions (e.g. is the Environmental Performance of a property within the allowed classes). **CertificateCheck** refers to regulations that are satisfied via the demonstration of appropriate certification (e.g. does an installer have certificates to demonstrate their status as a Competent Person in the UK regulations, or does a building component have the required CE marking). Finally, **HumanEvaluatedCheck** refers to checks that cannot be automated (e.g. is a building aesthetically acceptable for its surroundings), as human intervention and decision is required. In this latter case, implementations of the ontology can allow the production of respective flags to highlight to human users that this needs to be validated by them. The subclasses of **Check** are shown in Fig. 4.

While the default implementation of data inputs is via the **hasRequiredData** data property (Fig. 4), BCO also provides high-level support for using different file types as inputs, via the **RequiredMedium** class. This is implemented in order to allow access to common file types, such as Documents, Images, Drawings, and Models, which are defined and reused from existing ontologies. In this first version, widely used external ontologies are employed (DCMI for documents, images, and drawings, and ifcOWL for nD models), though it can be extended to accommodate other ontologies. The current structure is shown in Fig. 5.

The development of BCO was completed with the aim of answering the following competency questions:

- What does this piece of compliance regulation say?
- What compliance conditions does it set, and how are they satisfied?
- What data is needed from the building design documentation to perform the relevant checks or calculations?
- What is the source document for this statement, and what is precise reference to it?
- To which AEC disciplines does this statement pertain?
- To which types of building does this statement pertain?
- What is the legal status of this statement (e.g. mandatory, advisory...) for the currently examined jurisdiction?
- Who verifies this statement for building permitting purposes, at which stage of the process, and what evidence is required to demonstrate compliance?

The core ontology presented here provides the high-level architecture. In order to perform automated compliance checking for specific building regulations, implementation of appropriate, specific assertion component (ABox) is required. An example of lower-level implementation for Structural Engineering is presented in the next Section.

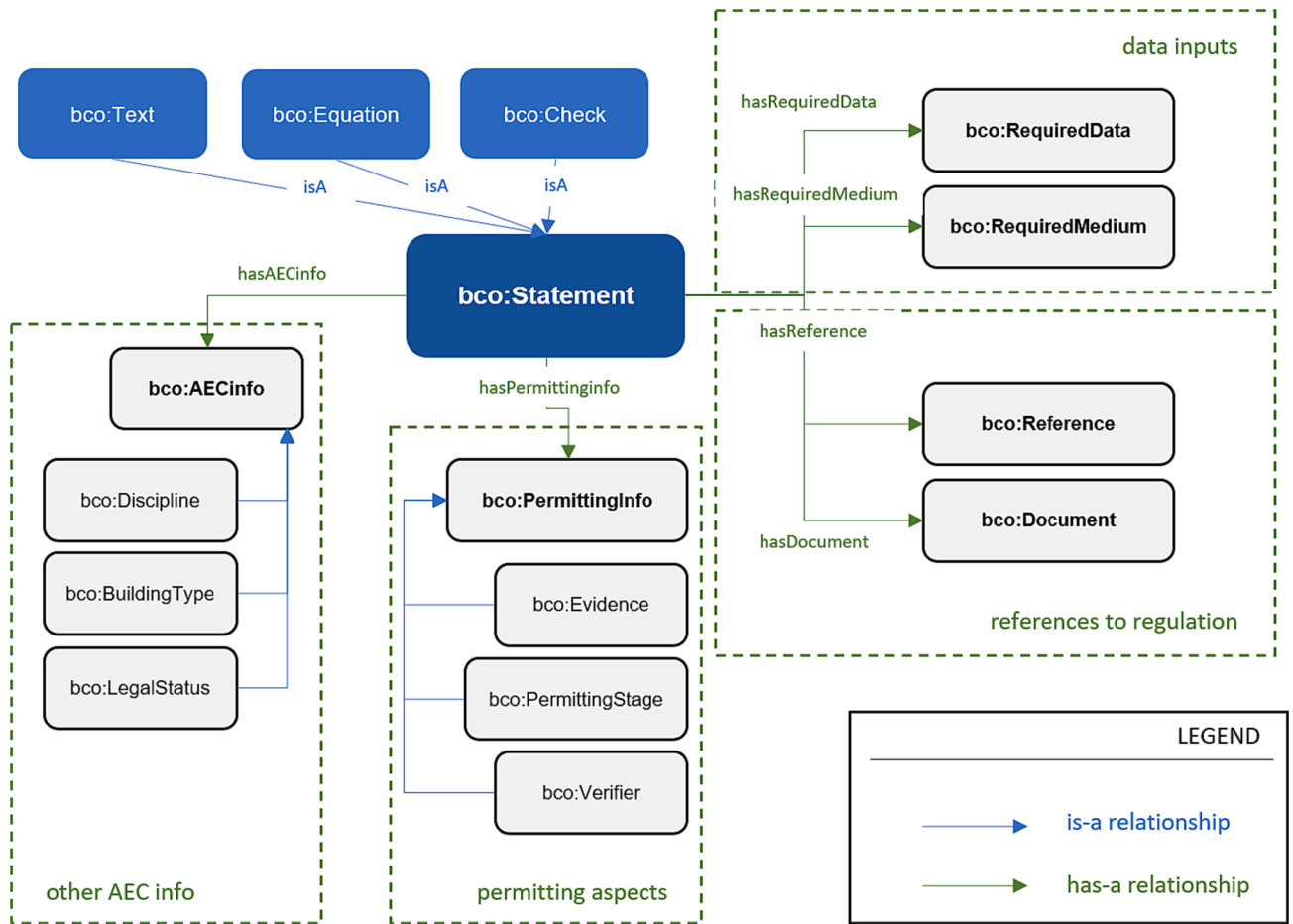


Fig. 3. Illustration of top-level BCO structure.

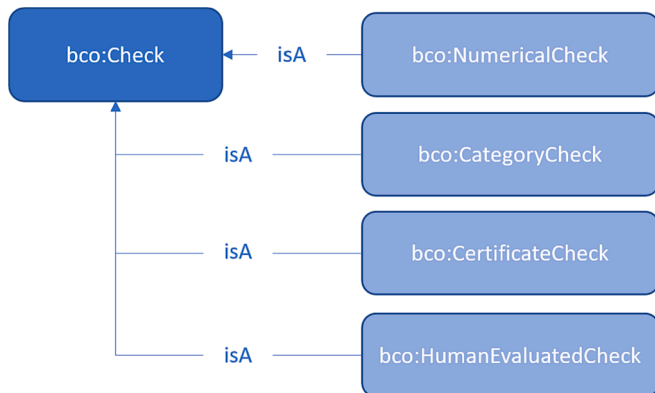


Fig. 4. Illustration of Check subclasses.

#### 4. Framework implementation for Automatic compliance checking in structural engineering

##### 4.1. Overview

Structural Engineering offers an ideal domain to examine the application of the framework presented above. The expectations of compliance in structural engineering are highly complex, highly detailed, and assume a high degree of expertise; they combine extensive mathematical calculations with qualitative assessments; they are a necessary part of all building designs, demonstrating the most fundamental safety aspect, structural integrity. When supplemented by Finite

Element Modelling (FEM), they require one of the most complex, and computationally demanding, processes in all of construction. As such, it is reasonable to posit that, if a framework can accommodate Structural Engineering requirements, it is likely to be able to accommodate other areas of compliance as well.

Applying the general process presented in Section 2 to solve Structural Engineering problems in the context of modular house construction, the following domain-specific results are derived:

An IFC model, containing geometry, material information, and section profiles is developed by the designers.

An open-source FEM solver, Code\_Aster, is utilised to perform Finite Element analysis of structural members. The IFC2CA tool, discussed in Section 4.2, is utilised to automate the I/O process between the IFC model and Code\_Aster.

The results of the FEM analysis are fed back to IFC2CA, and the IFC model is enriched with those.

Ontology instances are generated for the compliance checking, and intermediate calculations for optimisation are performed, using a middleware tool, IFC2BCO, also discussed in Section 4.2.

The ontology instances, discussed in Section 4.3, are introduced in Protégé, and compliance is checked automatically for relevant statements of the Building Regulations, using SHACL.

This is shown in Fig. 6.

The environments presented in Fig. 6 are not the only way to adhere to the process, but the tools chosen by the authors for the purposes of this work.

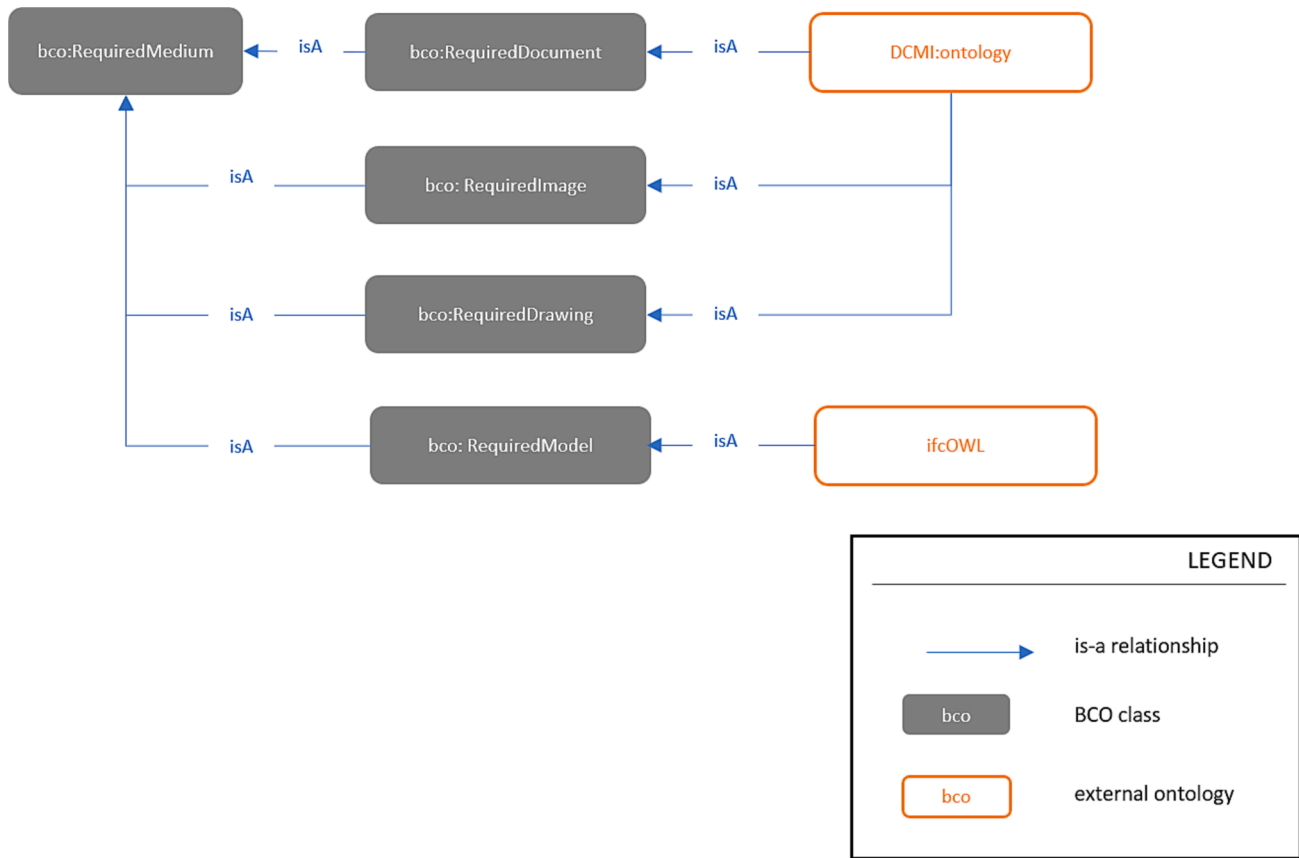


Fig. 5. RequiredMedium and its subclasses.

#### 4.2. The IFC2CA tool

Utilising computational structural analysis of a design, and successfully interpreting its outcomes is a key part of the process. As a general principle, the information contained within a correctly prepared and comprehensive structural IFC file, which can be generated using standard industry software such as Autodesk Revit or BlenderBIM, suffices as a foundation for conducting a complete structural analysis of a building with computational tools. Performing Finite Element Analysis (FEA) on the model typically culminates in the generation of field of outputs, encompassing displacements, generalized forces, and stresses associated with the discretized input for the analysis. These results need to be reintegrated into the IFC file, so the new and enriched version can be used for structural verification in the context of the appropriate structural design codes (in this case, the Eurocodes), by comparing design actions with design resistances. The FEM solver utilised for the purposes of this work is Code\_Aster, developed by EDF (Électricité [12]. As this provides a range of analysis options (including linear and nonlinear analyses, static and dynamic simulations, and thermal and fluid-structure interactions) the process described here is easily extendable to other types of complex computational analyses and domains. Code\_Aster also supports direct Python integration, allowing the writing of custom scripts to enhance the analysis efficiency and the accuracy of the results. The solver has been thoroughly validated through its use in a very wide range of engineering, industrial, and infrastructure projects, primarily, but not solely, associated with EDF, while it is open-source, which allows for a high level of transparency and replicability [9].

IFC2CA<sup>1</sup> is an open-source software developed by Aether Engineering, incorporated in the opensource project ifcopenshell, that enables the link between IFC models and Code\_Aster. In the first instance (IFC import block), IFC2CA is utilised to extract the appropriate information from an IFC file to perform FEA. This is achieved via the generation of

two scripts, each serving a different purpose:

- Salome script.** This script permits the creation of a geometric discretization (mesh) using the open-source software Salome[27]. It constructs and assigns unique names to geometries related to structural elements, structural connections, and structural loads, that can then be referenced by the solver during execution. The resulting mesh is exported in MED format, an open data format based on HDF5 specifications.
- Code\_Aster script.** This script, in.comm extension, is the command file for the FEA and contains Code\_Aster-specific Python commands. It is specifically designed to be executed in strict association with the mesh created by the Salome script. The Code\_Aster script encompasses the mechanics of the analysis, providing instructions for defining material properties, applying boundary conditions, specifying loads, and setting analysis parameters. As input to Code\_Aster, it ensures the accurate mechanical simulation of the structure and computation of the desired outputs to be extracted from the analysis solution, such as displacements, stresses, forces, or other relevant quantities of interest. In essence, it encapsulates the entire setup and configuration for the finite element analysis within Code\_aster, leveraging the information initially described in the IFC file.

The advantage of generating both Salome and Code\_Aster scripts is that the workflow for structural analysis is streamlined. The Salome script enables the creation of a suitable mesh, while the Code\_aster script provides the necessary instructions for conducting the finite element analysis and extracting the desired results. This integration enhances the efficiency and accuracy of the analysis process, ultimately aiding in the assessment and understanding of the structural behaviour of the building or construction project.

The FEA is executed leveraging a Docker implementation of

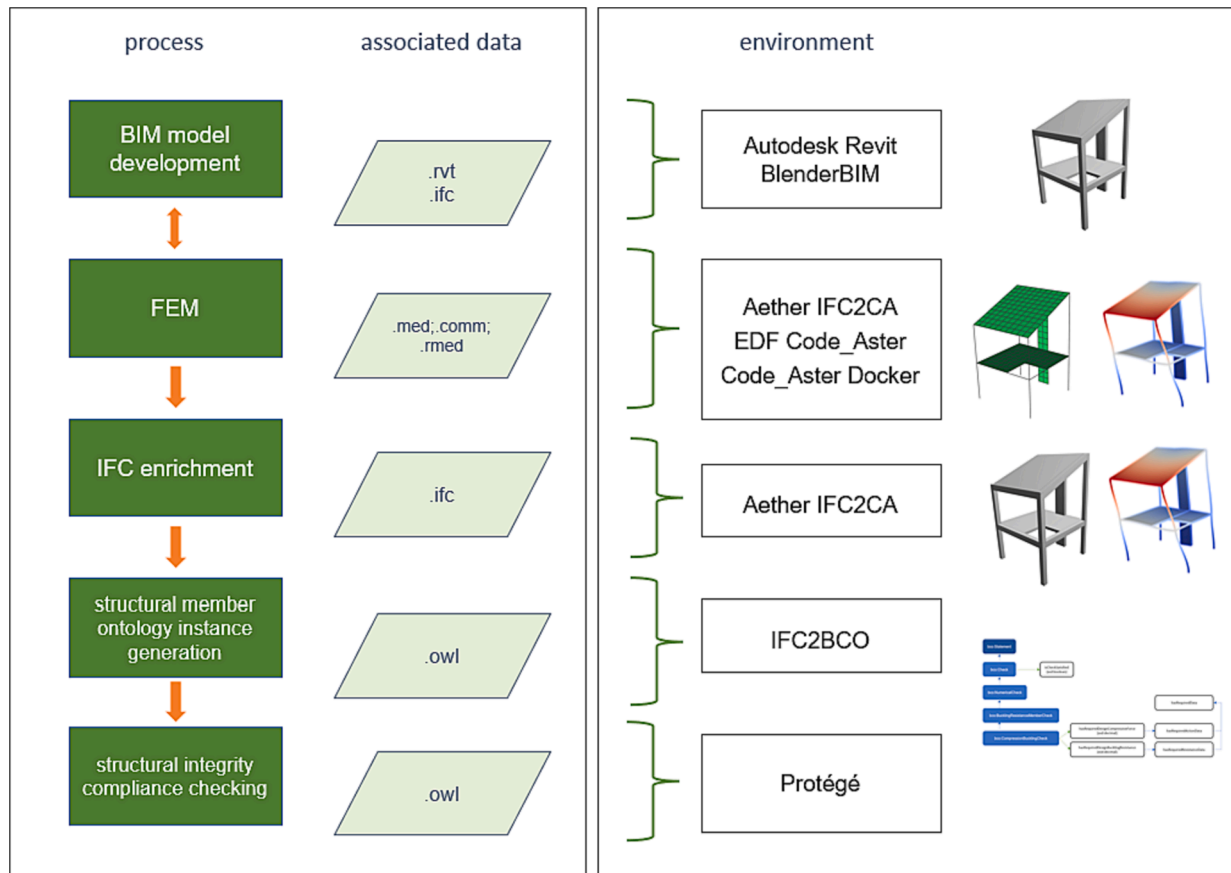


Fig. 6. Sample process for structural engineering analysis (as used in the Case Study).

Code\_Aster, also developed by Aether Engineering. Docker is the selected containerization platform to allow the packaging and deployment of software with a consistent and reproducible environment, regardless of the underlying infrastructure.

Code\_Aster executes the command file on the mesh and saves the results in MED format, incorporating an enhanced discretization of the result fields, from which tabular files with numerical results may be generated.

IFC2CA is utilised again in the IFC export block. In this step, IFC2CA is used to import the results of the FEA and enrich the initial IFC file with those. Initial parsing of the results is done with the use of the open-source software meshio, which are then directly associated to structural elements and connections in the IFC model and are written in the form of field results. This leads to an enriched model that contains all related analysis results.

The final step of the process requires the use of IFC2BCO. This is a middleware, purpose-developed by Aether Engineering in Python, whose function focuses on importing the enriched IFC file and exporting BCO instances, ready for validation. During this process, relevant regulations from the structural design codes (in this case the Eurocodes) are implemented, to minimise the computational load required during the ontology validation. Thus, the process is streamlined, and no complex mathematical operations are demanded by the ontology.

#### 4.3. BCO implementation for structural engineering

Utilising BCO for ACC in structural engineering involves primarily implementing equations from the respective structural design codes. As the case study is based on residential buildings with a steel frame in the UK, the relevant codes are Eurocode 0, Eurocode 1, and Eurocode 3, with emphasis on the latter. Drawing from the top-level ontology, most

design clauses in the Eurocodes involve **NumericalCheck** classes. An example of the implementation of the buckling check for members according to Clause 6.3.1.1 of Eurocode 3 (British Standards [7]), with the class inheritance structure and the associated data properties is given in Fig. 7 below.

The check also has the respective object properties, in line with the top-level structure given in Fig. 3, covering AEC info, Permitting info, and References to the source document, as presented in Fig. 8 below.

Basic SWRL rules can be used to assign such object properties en masse. For example, all classes that have Eurocode 3 as their source document can be assigned Structural Engineering as discipline and Steel Structure as building type.

#### 5. Application to a case study

The framework was tested via application to a case study. This was a lightweight structural steel modular house, developed by a UK-based cold-rolled lightweight steel manufacturer, which has a modular construction business using its own cold-rolled lightweight steel products. The manufacturer has its own offsite building systems. In this study, they provided information including an IFC model of a generic modular house model that they used for production purpose. This building system enables the adoption of either panelised or volumetric method for production and assembly, with cold-rolled lightweight steel profiles as structural members forming the building frame including the walls and floors. The generic modular house is designed to be adaptable to the typical UK low-rise residential typologies such as terraced, semi-detached and detached houses. This is considered a practically useful case study, as it combined a relatively straightforward structure, with a detailed IFC model (Figs. 9 and 10) used not only for design but also manufacturing purposes, while describing a system with wide



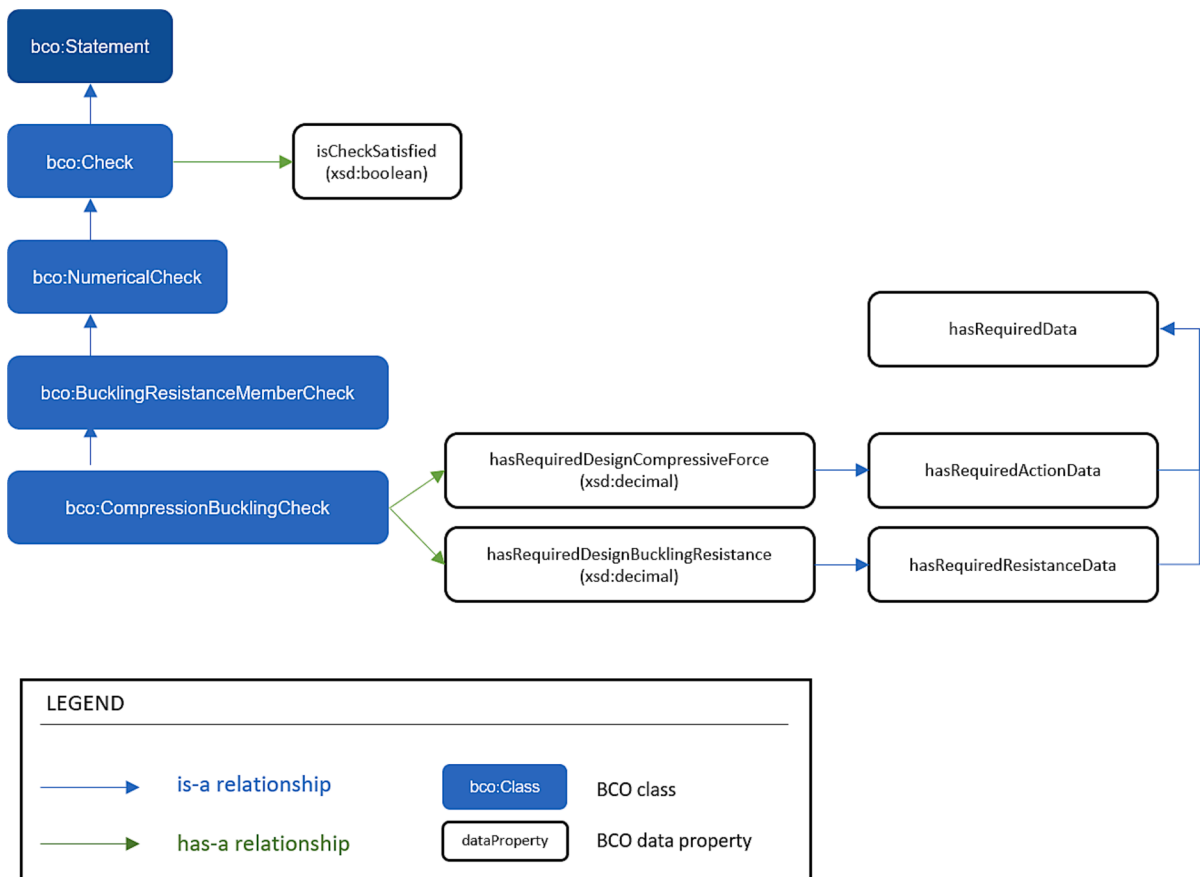


Fig. 7. Inheritance structure of the Compression Buckling Check class.

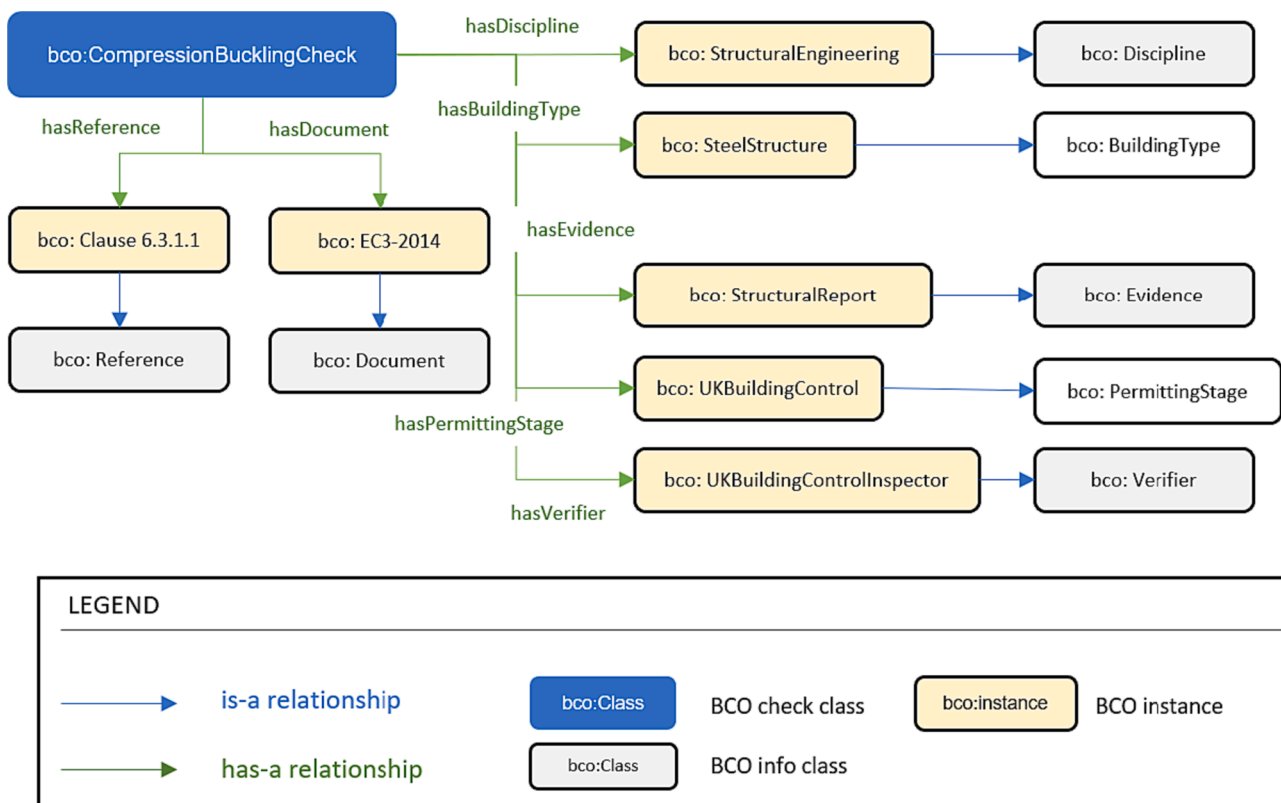


Fig. 8. Object properties of the Compression Buckling Check class.

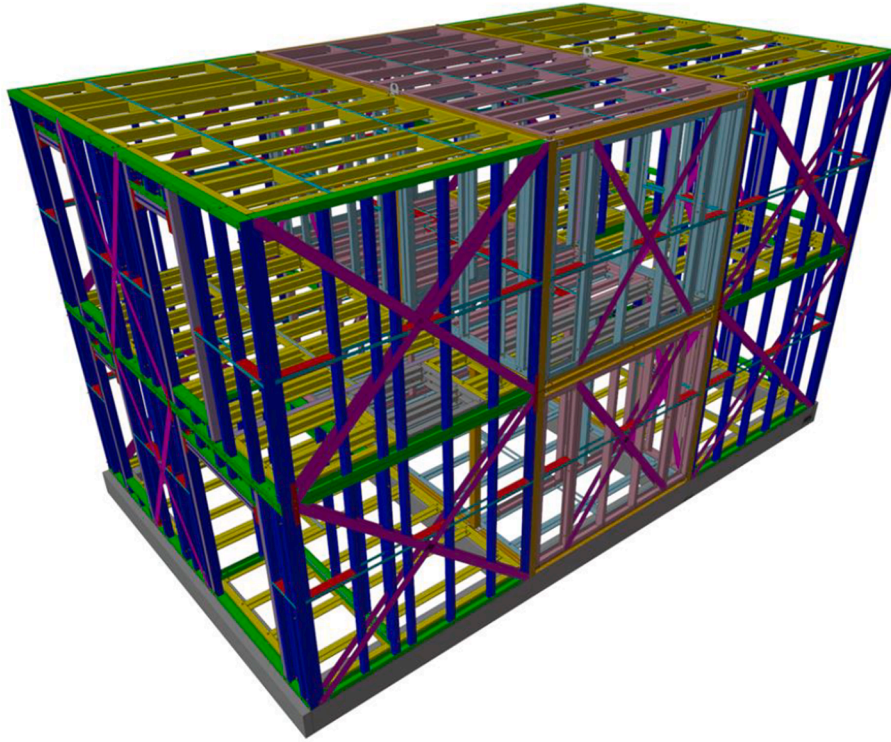


Fig. 9. Structural BIM model of the offsite module, with legend of member types.

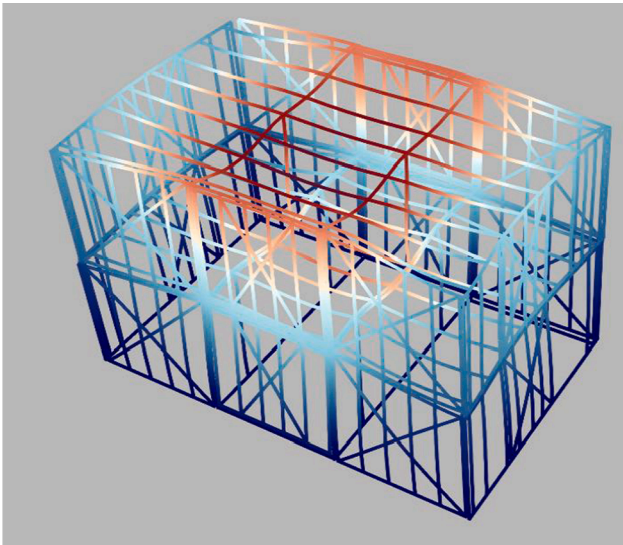


Fig. 10. BIM model of the core structural elements the offsite module, dimensions and legend.

applicability and adaptability.

Table 2 shows the types of steel profiles used in the offsite module. The dimensions of the two-storey building unit were 5.45 m x 9.15 m in plan with an inter-storey height of 2.6 m.

Table 3 shows the respective load cases, used in the Finite Element model.

In accordance with the process described in Section 4.1, IFC2CA's import block was utilised to load the IFC model into the Code\_Aster solver, where the Finite Element Analysis was undertaken. IFC2CA is executed in script form, and the results are returned automatically in MED format, a type of binary file format native to Code\_Aster, based on the Hierarchical Data Format (HDF). This means that no interaction with

Table 2

Schedule of Members of the Offsite Module.

| Element Type - Name | Material | Profile           | Description                  |
|---------------------|----------|-------------------|------------------------------|
| COLUMN-MAIN (14)    | S275     | SHS100 x 100 x 6  | Tubular 100 x 100 x 6 mm     |
| BEAM-MAIN (32)      | S450     | PFC-100 x 50 x 10 | C-shaped 100 x 50 x 5/8.5 mm |
| STUD (177)          | S450     | UZ-100F6012B      | C-shaped 100 x 60 x 1.2 mm   |
| STUD (154)          | S450     | UZ-100F6015B      | C-shaped 100 x 60 x 1.5 mm   |
| STUD (22)           | S450     | UZ-100F6020B      | C-shaped 100 x 60 x 2 mm     |
| ROOF JOIST (70)     | S450     | UZ-100F7512B      | C-shaped 100 x 75 x 1.2 mm   |
| JOIST (171)         | S450     | UZ-100F7520B      | C-shaped 100 x 75 x 2.0 mm   |
| LINTEL (16)         | S450     | UZ-100F5012B      | C-shaped 100 x 50 x 1.2 mm   |
| PLATE 62            | S450     | UZ-104T7012B      | U-shaped 104 x 70 x 1.2 mm   |
| PLATE 139           | S450     | UZ-104T7020B      | U-shaped 104 x 70 x 2.0 mm   |
| PLATE 17            | S450     | UZ-104T4012B      | U-shaped 104 x 40 x 1.2 mm   |
| BRACING (60)        | S450     | UZ-B100           | Rectangular 100 x 1.2 mm     |
| BRACING (116)       | S275-JR  | UZ-B38            | Rectangular 38 x 0.9 mm      |
| BEAM-PLATE 38       | S275-JR  | PLT3 x 90         | Rectangular 3 x 90 mm        |
| BEAM-FOUNDATION (6) | C30      | 350 x 350         | Rectangular 350 x 350 mm     |

a UI is required. For visualisation purposes, an open-source environment such as Salome-Meca can be used.

The numerical building model consisted of classical Euler beam elements, connected at joints with appropriate pinned or fixed node conditions specified at the end of each member. The joint selection depended on standard structural engineering modelling assumptions.

**Table 3**

Load cases used in the FEM, applicability to members, and numerical values.

| Load Case | Description                                                                                  | Applicability                                                                                                                        |
|-----------|----------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| LC-1: G   | Permanent G - gravity load from self-weight of structural and non-structural components (FZ) | Floor joists: 1.0 kPa x 0.54 m = 0.54 kN/m<br>Roof joists: 0.5 kPa x 0.45 m = 0.225 kN/m<br>Wall headers: 0.5 kPa x 2.6 m = 1.3 kN/m |
| LC-2: Q   | Floor Q (FZ)                                                                                 | Floor joists: 2.0 kPa x 0.54 m = 1.08 kN/m                                                                                           |
| LC-3: S   | Snow S (FZ)                                                                                  | Roof joists: 1.0 kPa x 0.45 m = 0.45 kN/m                                                                                            |
| LC-4: WX+ | Wind WX+ (FX)                                                                                | Walls @X+: 0.8 kPa x 2.6 m = 2.08 kN/m<br>Walls @X-: 0.4 kPa x 2.6 m = 1.04 kN/m                                                     |
| LC-5: WX- | Wind WX- (FX)                                                                                | Walls @X+: 0.4 kPa x 2.6 m = 1.04 kN/m<br>Walls @X-: 0.8 kPa x 2.6 m = 2.08 kN/m                                                     |
| LC-6: WY+ | Wind WY+ (FY)                                                                                | Walls @Y+: 0.8 kPa x 2.6 m = 2.08 kN/m<br>Walls @Y-: 0.4 kPa x 2.6 m = 1.04 kN/m                                                     |
| LC-7: WY- | Wind WY- (FY)                                                                                | Walls @Y+: 0.4 kPa x 2.6 m = 1.04 kN/m<br>Walls @Y-: 0.8 kPa x 2.6 m = 2.08 kN/m                                                     |

The four external core columns were fixed at the base, whereas connections among beams and columns, studs and plates in the walls were defined with pinned conditions. (Fig. 11).

A linear static analysis was conducted for the load case combinations defined in Table 2.

Table 4 shows the Loading Case Combinations deriving from those loads, following the Eurocodes.

For the calculation of the buckling design resistance: the buckling length was taken equal to the length of the element and an imperfection factor of 0.21 according to buckling curve type “a” was considered, following Eurocode 3.

Fig. 12 shows the deformed shape, axial forces, shear forces, and bending moments under Loading Case Combination 2.

Further information, including geometry assumptions, elastic material properties used in the numerical analysis, safety factors used in the loading combinations, and maxima of the design actions can be found in the Appendix.

Using IFC2CA's export block, the IFC model was enriched with the results of the FEM analysis.

For the purposes of the case study, two types of compliance checks were undertaken:

**Table 4**

Loads and respective safety factors, used per loading case combination.

| Load CaseCombination | LC-1: G | LC-2: Q | LC-3: S | LC-4: WX+ | LC-5: WX- | LC-6: WY+ | LC-7: WY- |
|----------------------|---------|---------|---------|-----------|-----------|-----------|-----------|
| LCC-1                | 1.35    | 1.50    | 1.50    | –         | –         | –         | –         |
| LCC-2                | 1.35    | 1.05    | 0.75    | 1.50      | –         | –         | –         |
| LCC-3                | 1.35    | 1.05    | 0.75    | –         | 1.50      | –         | –         |
| LCC-4                | 1.35    | 1.05    | 0.75    | –         | –         | 1.50      | –         |
| LCC-5                | 1.35    | 1.05    | 0.75    | –         | –         | –         | 1.50      |

- Member checks in the Ultimate Limit State (ULS). The ULS checks identified as applicable for this structure were bending checks and shear checks for the beams, and compression buckling checks for the columns.
- Parameter checks. In those, the compliance check verified that appropriate values for Eurocode parameters, such as the partial factor for the design resistance, were used in the calculations.

An extension of BCO was developed in Protégé, implementing the relevant parts of the regulation. This was supplemented by the development of SHACL rules for validation. A snippet of a sample SHACL rule is given below:

Listing 1. A simple SHACL rule for the verification of compliance to shear resistance

```
# Check that the design shear force is less than or equal to the design shear resistance
bco:ShearShape
  a sh:NodeShape;
  sh:targetClass bco:ShearCheck; # Applies to all bending instances
  sh:property [
    sh:path bco:hasRequiredDesignShearForce;
    sh:lessThanOrEquals bco:hasRequiredDesignShearResistance;
    sh:message“Shear failure!”;
    sh:severitysh:Warning;
  ];
```

This ontology, together with the IFC model, were loaded to IFC2BCO, which then automatically generated the relevant Check instances. These instances were subsequently loaded back to Protégé. By running the SHACL script, the automatic compliance check to Eurocode 3 is performed automatically. A screenshot of the validation report is given in Fig. 13.

## 6. Conclusion

The work presented in this paper is the first, to our knowledge, attempt in the field of Automatic Compliance Checking to develop a comprehensive process that combines Finite Element Modelling with a Semantic Web-based verification platform in a unified workflow, in order to automatically check compliance of some structural engineering requirements. The process was tested on a real-world design: a structural model of a modular steel structure, suitable for offsite manufacturing. A

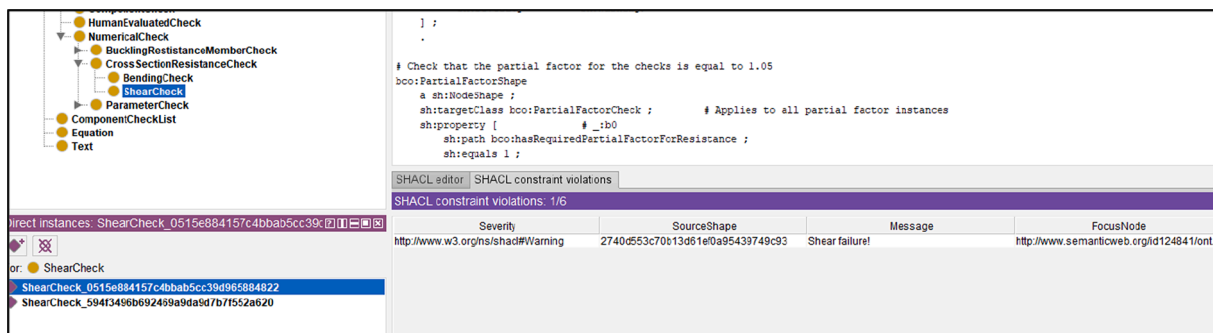


Fig. 11. Visualisation of the FEM geometry. The geometry is automatically created by the IFC2CA block.

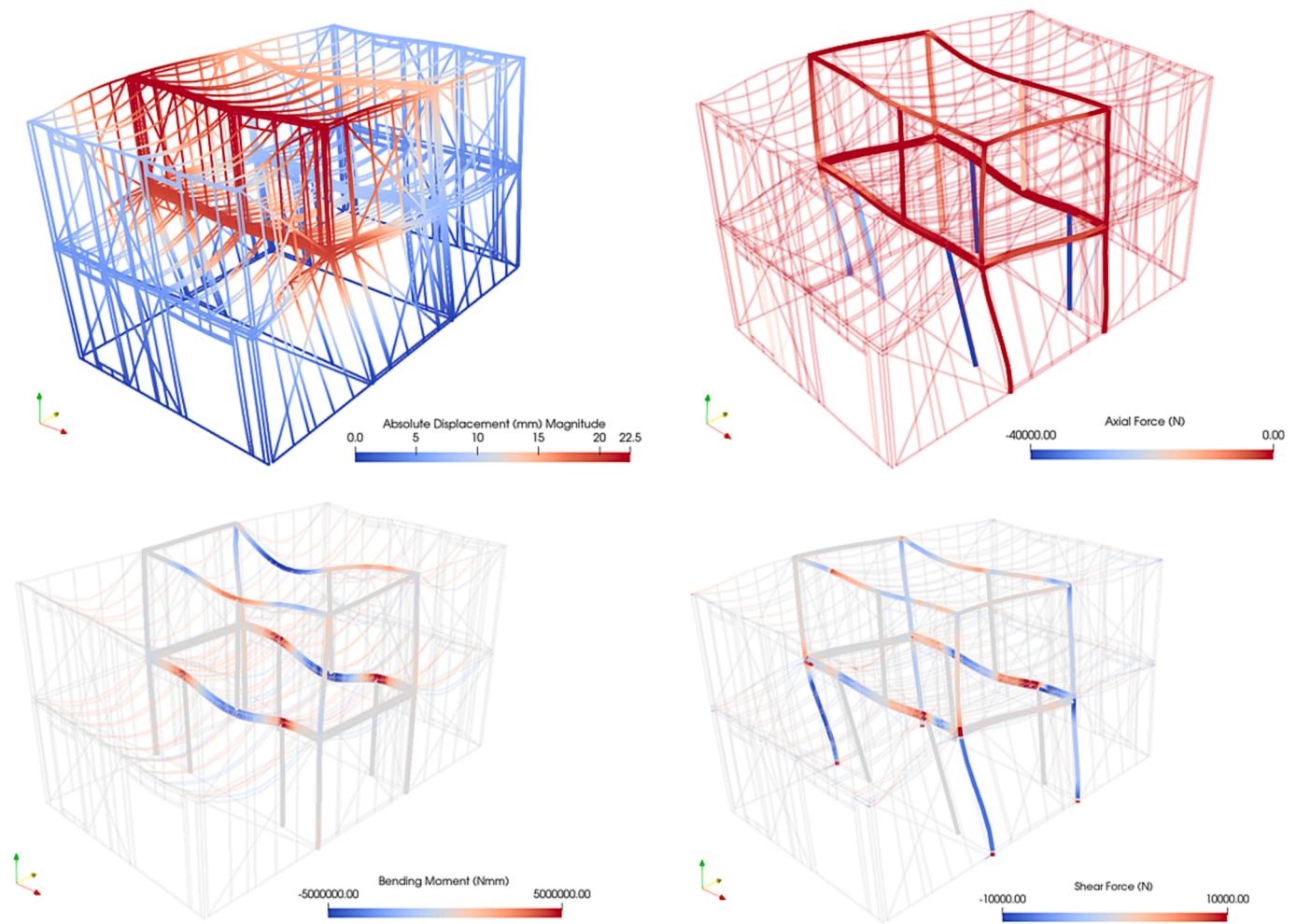


Fig. 12. Outputs of the FEA for Loading Case Combination 2. Clockwise from top left: Absolute Deformation (in mm), Axial Force (in N), Shear Force (in N), Bending Moment (in Nmm).

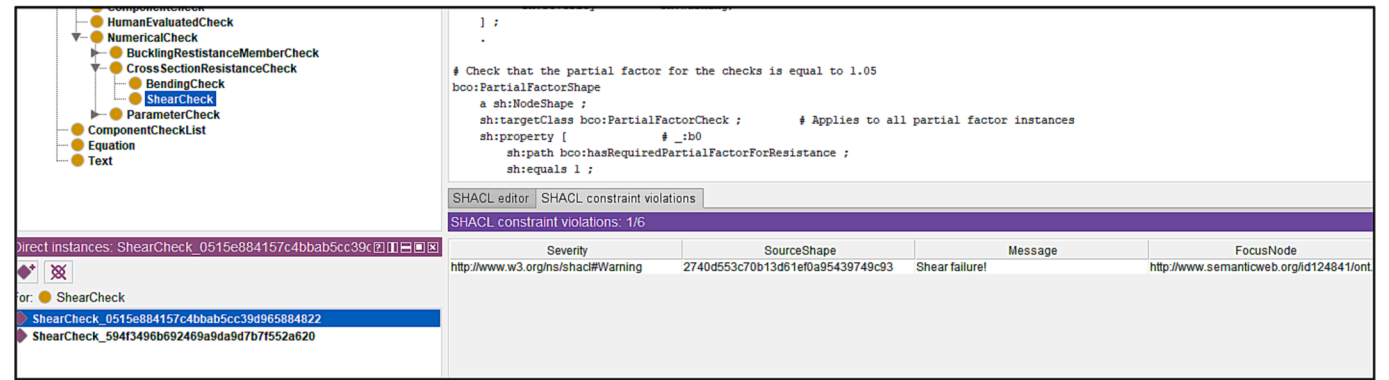


Fig. 13. An example of a validation report with a member failing in resistance against shear.

BIM model in the IFC format was used as a starting point. The system was successful in automatically performing a Finite Element Analysis (FEA) of the design, enriching the IFC model with the FEA results, and finally checking compliance for member design automatically via a Semantic Web-oriented platform.

There are three important limitations in the work presented here, which affect its immediate applicability in an industry context. Firstly, the workflow was implemented for, and validated in, the structural engineering domain only. The generic workflow described in subsection

2.2 and the top-level ontology presented in Section 3 have been conceived as generic and flexible, with the potential of accommodating most building regulations, from any aspect of the AEC domain. However, validation in different domains will be required to prove this is indeed the case.

Secondly, while the structural analysis and the structural compliance checking have been automated, the connection between the different elements of the system, operating on different platforms, still required some human intervention. The assumption that has been made is that



developing a platform that connects the different systems and executes them automatically is a software engineering exercise with little research interest that does not further the case study by itself. However, from a commercial perspective, developing a system that integrates and executes all the steps in Fig. 6 in “a single click” format is resource- and time-intensive process.

Thirdly, and most importantly, only a small subset of regulations has been implemented. The number of regulations by itself is not a critical issue, as the work was intended as a feasibility study and not a commercial software platform. However, the type of regulations automated is an important limitation. The regulations chosen were numerical based, with few unknowns, and little room for interpretation. As such, they were the most straightforward to implement, both with regard to extracting the required data and with regard to identifying a clear, definite benchmark against which compliance could be judged. There are many other types of AEC regulations, including regulations that require human judgement, whose automation might be substantially more difficult or even impossible. There are also substantial, well-documented technical challenges around the extraction of relevant information from BIM models.

The offsite sector was considered a good case study in this case as the modular, repetitive nature of the design work often means that the parameters are much fewer, and more easily controlled, compared to traditional, “one-off” construction. It might well be the case that offsite and modular construction will be the first sector to achieve ACC.

Next steps for this line of research involve primarily two aspects. The first is developing the tools to integrate other computationally-intensive approaches, such as CFD. The second is implementing more complex checks, such as Category Checks and Certificate Checks, while engaging with multiple input formats, with the eventual aim of developing a unified ACC platform, which would combine Digital Twin and Semantic Web principles, aiming to be agnostic with regard to input format.

#### CRediT authorship contribution statement

**Panagiotis Patlakas:** Conceptualization, Formal analysis, Investigation, Methodology, Validation, Visualization, Writing – original draft. **Ioannis Christovasilis:** Formal analysis, Methodology, Software, Visualization, Writing – review & editing. **Lorenzo Riparbelli:** Writing – review & editing. **Franco KT Cheung:** Funding acquisition, Project administration, Supervision, Writing – review & editing. **Edlira Vakaj:** Conceptualization, Funding acquisition, Project administration, Writing – review & editing.

#### Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: [This paper forms part of the outcomes of the ACCORD project, funded by the European Union’s Horizon Europe research and innovation programme (grant agreement no. 101056973). Specifically, this study has received support from the UKRI (no. 10038999), provided to Birmingham City University.].

#### Data availability

Data will be made available on request.

#### Acknowledgements

This paper forms part of the outcomes of the ACCORD project, funded by the European Union’s Horizon Europe research and innovation programme (grant agreement no. 101056973). Specifically, this study has received support from the UKRI (no. 10038999), provided to Birmingham City University.

#### References

- [1] R. Amor, J. Dimyadi, The promise of automated compliance checking. *Developments in the Built Environment* 5 (2021) 100039, <https://doi.org/10.1016/j.dibe.2020.100039>.
- [2] Autodesk. (n.d.). *Building Information Modelling*. Retrieved November 28, 2023, from <https://www.autodesk.com/solutions/aec/bim>.
- [3] Autodesk. (2023). *What Is a Digital Twin? How Intelligent Data Models Can Shape the Built World*. <https://www.autodesk.com/design-make/articles/what-is-a-digital-twin>.
- [4] K.O. Ayinla, F. Cheung, A.-R. Tawil, Demystifying the concept of offsite manufacturing method, *Constr. Innov.* 20 (2) (2020) 223–246, <https://doi.org/10.1108/CI-07-2019-0064>.
- [5] L.L. Bathurst, Automated building design, in: *Proceedings of the SHARE Design Automation Project*, 1965, pp. 11–17.
- [6] British Standards Institution. (n.d.). *What is BIM? (Building Information Modelling)*. Retrieved November 28, 2023, from <https://www.bsigroup.com/en-GB/Building-Information-Modelling-BIM/>.
- [7] British Standards Institution. (2014). *Eurocode 3 : design of steel structures. Part 1-1, General rules and rules for buildings*.
- [8] M. Charlier, A. Glorieux, X. Dai, N. Alam, S. Welch, J. Anderson, O. Vassart, A. Nadjai, Travelling fire experiments in steel-framed structure: numerical investigations with CFD and FEM, *Journal of Structural Fire Engineering* 12 (3) (2021) 309–327.
- [9] I.P. Christovasilis, L. Riparbelli, G. Rinaldin, G. Tamagnone, Methods for practice-oriented linear analysis in seismic design of cross laminated timber buildings, *Soil Dyn. Earthq. Eng.* 128 (2020) 105869, <https://doi.org/10.1016/j.soildyn.2019.105869>.
- [10] Digital Twin consortium. (2023). *What is a Digital Twin?* <https://www.digitaltwinconsortium.org/initiatives/the-definition-of-a-digital-twin/>.
- [11] C. Eastman, J. Lee, Y. Jeong, J. Lee, Automation in construction automatic rule-based checking of building designs, *Autom. Constr.* 18 (8) (2009).
- [12] Électricité De France. (2022). *Analyse des Structures et Thermo-mécanique pour des Etudes et Recherches*. In *Manuel d'Utilisation*.
- [13] V.R. Gharehbaghi, E. Noroozinejad Farsangi, M. Noori, T.Y. Yang, S. Li, A. Nguyen, C. Málaga-Chuquitaype, P. Gardoni, S. Mirjalili, A critical review on structural health monitoring: definitions, methods, and perspectives, *Arch. Comput. Meth. Eng.* 29 (4) (2022) 2209–2235, <https://doi.org/10.1007/s11831-021-09665-9>.
- [14] Hamil, S. (2021, September 9). *What is Building Information Modelling*. <https://www.thenbs.com/knowledge/what-is-building-information-modelling-bim>.
- [15] ISO/TC 59/SC 13. (2018). *ISO 19650-1: 2018 Organization and digitization of information about buildings and civil engineering works, including building information modelling (BIM) Part 1: Concepts and principles*.
- [16] L. Jiang, J. Shi, C. Wang, Multi-ontology fusion and rule development to facilitate automated code compliance checking using BIM and rule-based reasoning, *Adv. Eng. Inf.* 51 (2022) 101449, <https://doi.org/10.1016/j.aei.2021.101449>.
- [17] I.B. Kjtarsdóttir, S. Mordue, P. Nowak, D. Philp, J.T. Snæbjörnsson, *Building information modelling-BIM*, Civil Engineering Faculty of Warsaw University of Technology Warsaw, Poland, 2017.
- [18] S. Malsane, J. Matthews, S. Lockley, P.E.D. Love, D. Greenwood, Development of an object model for automated compliance checking, *Autom. Constr.* 49 (PA) (2015) 51–58, <https://doi.org/10.1016/j.autcon.2014.10.004>.
- [19] D. Mariano-Hernández, L. Hernández-Callejo, A. Zorita-Lamadrid, O. Duque-Pérez, F.S. García, A review of strategies for building energy management system: model predictive control, demand side management, optimization, and fault detect & diagnosis, *Journal of Building Engineering* 33 (2021) 101692.
- [20] S. Moon, G. Lee, S. Chi, Automated system for construction specification review using natural language processing, *Adv. Eng. Inf.* 51 (2022) 101495, <https://doi.org/10.1016/j.aei.2021.101495>.
- [21] N. Morozova, F.X. Trias, R. Capdevila, C.D. Pérez-Segarra, A. Oliva, On the feasibility of affordable high-fidelity CFD simulations for indoor environment design and control, *Build. Environ.* 184 (2020) 107144, <https://doi.org/10.1016/j.buildenv.2020.107144>.
- [22] F. Noardo, T. Wu, K. Arroyo Ohori, T. Krijnen, J. Stoter, IFC models for semi-automating common planning checks for building permits, *Autom. Constr.* 134 (2022) 104097, <https://doi.org/10.1016/j.autcon.2021.104097>.
- [23] P. Patlakas, A. Livingstone, R. Hairstans, G. Neighbour, Automatic code compliance with multi-dimensional data fitting in a BIM context, *Adv. Eng. Inf.* 38 (2018) 216–231, <https://doi.org/10.1016/j.aei.2018.07.002>.
- [24] P. Pauwels, A. Roxin, in: *SimpleBIM: from Full ifcOWL Graphs to Simplified Building Graphs*, CRC Press, 2017, pp. 11–18.
- [25] J. Peng, X. Liu, Automated code compliance checking research based on BIM and knowledge graph, *Sci. Rep.* 13 (1) (2023) 7065, <https://doi.org/10.1038/s41598-023-34342-1>.
- [26] M.H. Rasmussen, M. Lefrançois, G.F. Schneider, P. Pauwels, BOT: the building topology ontology of the W3C linked building data group, *Semantic Web* 12 (1) (2021) 143–161.
- [27] Ribes, A., Caremoli, C. (2007). Salome platform component model for numerical simulation. *31st Annual International Computer Software and Applications Conference (COMPSAC 2007)*, 2, 553–564.
- [28] D.M. Salama, N.M. El-Gohary, Semantic text classification for supporting automated compliance checking in construction, *J. Comput. Civ. Eng.* 30 (1) (2016) 04014106, [https://doi.org/10.1061/\(ASCE\)CP.1943-5487.0000301](https://doi.org/10.1061/(ASCE)CP.1943-5487.0000301).
- [29] A.Z. Sampaio, A.M. Gomes, BIM interoperability analyses in structure design, *CivilEng* 2 (1) (2021) 174–192.



- [30] S. Shirowzhan, S.M.E. Sepasgozar, D.J. Edwards, H. Li, C. Wang, BIM compatibility and its differentiation with interoperability challenges as an innovation factor, *Autom. Constr.* 112 (2020) 103086, <https://doi.org/10.1016/j.autcon.2020.103086>.
- [31] E. Vakaj, F. Cheung, J. Cao, A.-R. H. Tawil, P. Patlakas, An ontology-based cost estimation for offsite construction, *Journal of Information Technology in Construction* 28 (2023) 220–245, <https://doi.org/10.36680/j.itcon.2023.011>.
- [32] Y. Wang, M. Kyriakidis, V.N. Dang, Incorporating human factors in emergency evacuation – an overview of behavioral factors and models, *Int. J. Disaster Risk Reduct.* 60 (2021) 102254, <https://doi.org/10.1016/j.ijdrr.2021.102254>.
- [33] J. Werbrouck, P. Pauwels, J. Beetz, L. van Berlo, Towards a decentralised common data environment using linked building data and the solid ecosystem, 36th CIB, 2019.
- [34] O. Wong Chong, J. Zhang, R.M. Voyles, B.C. Min, BIM-based simulation of construction robotics in the assembly process of wood frames, *Autom. Constr.* 137 (2022) 104194, <https://doi.org/10.1016/J.AUTCON.2022.104194>.
- [35] T. Wu, A. Khan, M. Yong, G. Qi, M. Wang, Efficiently embedding dynamic knowledge graphs, *Knowl.-Based Syst.* 250 (2022) 109124, <https://doi.org/10.1016/j.knosys.2022.109124>.
- [36] Z. Xu, T. Zayed, Y. Niu, Comparative analysis of modular construction practices in mainland China, Hong Kong and Singapore, *J. Clean. Prod.* 245 (2020) 118861, <https://doi.org/10.1016/J.JCLEPRO.2019.118861>.
- [37] J. Zhang, A.M. Asce, N.M. El-Gohary, Semantic-Based logic representation and reasoning for automated regulatory compliance checking, *J. Comput. Civ. Eng.* (2016), [https://doi.org/10.1061/\(ASCE\)](https://doi.org/10.1061/(ASCE)).
- [38] J. Zhang, N.M. El-Gohary, Integrating semantic NLP and logic reasoning into a unified system for fully-automated code checking, *Autom. Constr.* 73 (2017) 45–57, <https://doi.org/10.1016/J.AUTCON.2016.08.027>.
- [39] B. Zhong, C. Gan, H. Luo, X. Xing, Ontology-based framework for building environmental monitoring and compliance checking under BIM environment, *Build. Environ.* 141 (2018) 127–142, <https://doi.org/10.1016/J.BUILDENV.2018.05.046>.
- [40] Y.C. Zhou, Z. Zheng, J.R. Lin, X.Z. Lu, Integrating NLP and context-free grammar for complex rule interpretation towards automated compliance checking, *Comput. Ind.* 142 (2022) 103746, <https://doi.org/10.1016/J.COMPIND.2022.103746>.